

Advanced Algorithms - Mock Exams

Includes three practice exams (MCQ + True/False with negative marking), plus answer keys and brief explanations.

Assumed T/F scoring style: +1 correct, -1 wrong, 0 blank.

Mock Exam 1

A) Multiple Choice (20 questions)

MCQ1 [Easy] The divide-and-conquer strategy is best described as:

- A) Solve the whole problem directly with a single pass
- B) Reduce the problem to a different domain and solve once
- C) Recursively split into smaller subproblems, solve them, then combine
- D) Randomly sample the input to estimate the output

MCQ2 [Medium] Strassen's matrix multiplication improves over the classical algorithm mainly by:

- A) Reducing the number of additions
- B) Reducing the number of multiplications in the recursive step
- C) Using hashing to avoid multiplications
- D) Switching to greedy selection of pivots

MCQ3 [Medium] In the Fibonacci example (counting additions), the naive recursive approach leads to:

- A) Constant number of additions
- B) Linear number of additions
- C) Exponential number of additions
- D) Always exactly $n-1$ additions

MCQ4 [Medium] Memoization improves a recursive solution primarily by:

- A) Sorting subproblems so they are solved in increasing order
- B) Caching previously computed results to avoid repeated work
- C) Replacing recursion with random sampling
- D) Guaranteeing optimality even when the recurrence is wrong

MCQ5 [Medium] A hallmark of dynamic programming (DP) is that it:

- A) Works only for graph problems
- B) Requires randomness to reduce variance
- C) Exploits overlapping subproblems and stores intermediate results
- D) Avoids recursion entirely (DP cannot be implemented recursively)

MCQ6 [Medium] In the Coin Row Problem, a standard DP approach needs an extra step if you want the actual chosen coins, not just the optimal value:

- A) A second pass that traces back choices (or stores decisions as you compute)
- B) A different hash function per coin
- C) A Bloom filter to test membership of coins
- D) A Count-Min Sketch to estimate coin values

MCQ7 [Medium] Why does a greedy strategy that works for the continuous/fractional knapsack fail for the 0-1 knapsack?

- A) Greedy is never correct for any knapsack variant
- B) Fractional knapsack allows taking fractions, which makes value/weight locally optimal choices globally consistent
- C) 0-1 knapsack is polynomial-time solvable by greedy if you pick the right tie-breaker
- D) Fractional knapsack requires backtracking, 0-1 does not

MCQ8 [Medium] Which pair of algorithms are classic greedy graph algorithms?

- A) Prim and Kruskal
- B) Knapsack DP and Coin Row DP
- C) Flajolet-Martin and HyperLogLog
- D) Misra-Gries and Count-Min

MCQ9 [Medium] Prim's algorithm grows a minimum spanning tree mainly by repeatedly:

- A) Picking the globally smallest edge in the entire graph regardless of connectivity
- B) Picking the cheapest edge that connects the current tree to a new vertex
- C) Removing cycles by decrementing all counters
- D) Replacing items using reservoir sampling

MCQ10 [Medium] Kruskal's algorithm builds a minimum spanning tree mainly by:

- A) Starting from one vertex and expanding outward
- B) Sorting edges by weight and adding edges that don't create a cycle
- C) Using DP over subsets of edges
- D) Hashing vertices into buckets and taking the minimum bucket

MCQ11 [Hard] In the limitations/coping part (backtracking + hard problems), backtracking is best characterised as:

- A) Always polynomial-time because it prunes paths early
- B) Systematic search over a solution space, possibly pruning partial solutions that can't lead to feasible/optimal solutions
- C) A randomized method that may return wrong answers
- D) A streaming method that never stores elements explicitly

MCQ12 [Hard] A statement that correctly matches the slide-level meaning of NP-complete is:

- A) Problems solvable in polynomial time with a greedy algorithm
- B) Problems in NP to which every problem in NP can be reduced in polynomial time
- C) Problems solvable with a Bloom filter and universal hashing
- D) Problems whose solutions can't be verified efficiently

MCQ13 [Medium] SAT appears in your slides mainly as:

- A) A heuristic for TSP
- B) A classic reference problem in NP-completeness discussions
- C) A data-stream primitive for frequency estimation
- D) A method to remove collisions in Count-Min

MCQ14 [Medium] A Las Vegas randomized algorithm is one that:

- A) Always runs in worst-case polynomial time but may return a wrong answer
- B) Always returns a correct answer, but its runtime is random
- C) Uses no random bits, only worst-case analysis
- D) Always returns an approximate answer faster than exact methods

MCQ15 [Medium] A Monte Carlo randomized algorithm is one that:

- A) Always correct but runtime random
- B) Always correct and runtime fixed
- C) May return an incorrect answer with some probability, typically with bounded runtime
- D) Requires DP to be applicable

MCQ16 [Hard] Reservoir sampling is used when:

- A) The full input of size n is known in advance and stored in memory
- B) You need a uniform sample from a stream when n is not known in advance
- C) You need exact set membership with zero false positives
- D) You want to compute a minimum spanning tree

MCQ17 [Hard] In the approximate counting with arbitrary base a (probabilistic counters): if the counter value is k , the update rule is:

- A) Increment with probability $1/k$
- B) Increment with probability $1/a^k$
- C) Increment with probability a^k
- D) Always increment (deterministic)

MCQ18 [Hard] In the same approximate counting method, an estimate of the number of events from counter value k is:

- A) $(a^k - a + 1)/(a - 1)$

- B) $a^k(a - 1)$
- C) k/a
- D) $k(a - 1)$

MCQ19 [Medium] Bloom filters:

- A) Can produce false negatives but never false positives
- B) Can produce false positives but never false negatives (for standard insertion-only membership)
- C) Always store the full set explicitly
- D) Return exact frequencies of elements in a stream

MCQ20 [Medium] In the Count-Min Sketch, the query estimate for item i is typically:

- A) The maximum counter across rows at the hashed positions
- B) The minimum counter across rows at the hashed positions
- C) The average counter across rows at the hashed positions
- D) A random counter chosen from one row

B) True/False (20 statements)

TF1 [Easy] Divide-and-conquer solves subproblems recursively and then combines their solutions.

TF2 [Medium] Strassen's algorithm reduces the asymptotic complexity of matrix multiplication by reducing the number of recursive multiplications.

TF3 [Medium] Memoization is a technique that avoids recomputation by storing results of previous function calls.

TF4 [Hard] Memoization guarantees that a naive recursive Fibonacci becomes constant time.

TF5 [Medium] In DP, you typically store solutions to subproblems so later subproblems can reuse them.

TF6 [Medium] In the Coin Row Problem, computing only the optimal value is enough to recover the chosen coins without any additional information.

TF7 [Medium] A greedy rule that is optimal for fractional knapsack is not necessarily optimal for 0-1 knapsack.

TF8 [Medium] Prim and Kruskal are both greedy algorithms for minimum spanning trees.

TF9 [Hard] Kruskal's algorithm can add an edge even if it creates a cycle, as long as the edge is light.

TF10 [Medium] Backtracking may still take exponential time in the worst case.

TF11 [Hard] "NP-complete" means "not solvable, even approximately."

TF12 [Medium] SAT is commonly used as a reference problem when discussing NP-completeness.

TF13 [Medium] A Las Vegas algorithm may have random runtime but (when it outputs) it is correct.

TF14 [Medium] A Monte Carlo algorithm can have bounded runtime but may be wrong with some probability.

TF15 [Hard] Reservoir sampling is unnecessary if the stream length is unknown.

TF16 [Hard] In approximate counting with base a , the probability of increment decreases as the counter value increases.

TF17 [Medium] A standard Bloom filter has no false negatives (assuming no deletions and correct implementation).

TF18 [Hard] Bloom filters return exact membership, so they have neither false positives nor false negatives.

TF19 [Medium] Count-Min Sketch estimates are biased upward because collisions only increase counters.

TF20 [Hard] In Count-Min Sketch, taking the minimum across rows helps reduce the effect of collisions.

C) Answer Key

MCQ: 1C, 2B, 3C, 4B, 5C, 6A, 7B, 8A, 9B, 10B, 11B, 12B, 13B, 14B, 15C, 16B, 17B, 18A, 19B, 20B

T/F: 1T, 2T, 3T, 4F, 5T, 6F, 7T, 8T, 9F, 10T, 11F, 12T, 13T, 14T, 15F, 16T, 17T, 18F, 19T, 20T

D) Brief Explanations

MCQ1: C - Divide-and-conquer splits, solves recursively, then combines; the others describe different paradigms.

MCQ2: B - Strassen reduces the number of multiplications per recursive step; that's what improves asymptotics.

MCQ3: C - Naive Fibonacci repeats subcalls, producing an exponential-size recursion tree.

MCQ4: B - Memoization caches results so repeated calls become constant-time lookups.

MCQ5: C - DP stores solutions to overlapping subproblems; it is not limited to graphs and can be recursive.

MCQ6: A - To recover the chosen coins you must store/trace decisions; the value-only DP is insufficient.

MCQ7: B - Fractional knapsack allows splitting items, making the ratio rule globally valid; 0-1 does not.

MCQ8: A - Prim and Kruskal are the classic greedy MST algorithms.

MCQ9: B - Prim repeatedly picks the cheapest edge that expands the current tree to a new vertex.

MCQ10: B - Kruskal sorts edges and adds them if they do not create a cycle.

MCQ11: B - Backtracking explores a solution space systematically and prunes when partial solutions cannot work.

MCQ12: B - NP-complete = in NP and NP-hard via polynomial-time reductions.

MCQ13: B - SAT is a canonical NP-completeness reference problem.

MCQ14: B - Las Vegas: always correct, runtime random.

MCQ15: C - Monte Carlo: bounded runtime, small probability of error.

MCQ16: B - Reservoir sampling provides a uniform sample from a stream when n is unknown.

MCQ17: B - Update probability is $1/a^k$, decreasing as the counter grows.

MCQ18: A - The estimator maps stored k to an approximate large count: $(a^k - a + 1)/(a - 1)$.

MCQ19: B - Bloom filters have false positives but not false negatives in insertion-only mode.

MCQ20: B - Count-Min estimates use the minimum across rows at the hashed positions.

TF1: True - This is the definition of divide-and-conquer.

TF2: True - The asymptotic gain comes from fewer multiplications.

TF3: True - Memoization stores prior results to avoid recomputation.

TF4: False - Memoization makes Fibonacci about linear time, not constant.

- TF5:** True - DP relies on reusing stored subproblem solutions.
- TF6:** False - You need backpointers/traceback or stored decisions to reconstruct the chosen coins.
- TF7:** True - The greedy rule for fractional knapsack does not generally solve 0-1 knapsack.
- TF8:** True - Both Prim and Kruskal are greedy MST algorithms.
- TF9:** False - Kruskal must reject edges that create cycles.
- TF10:** True - Backtracking can still be exponential in worst case.
- TF11:** False - NP-complete is about reductions/verification, not impossibility or non-approximability.
- TF12:** True - SAT is the standard anchor problem for NP-completeness discussions.
- TF13:** True - Las Vegas algorithms are correct; randomness affects runtime.
- TF14:** True - Monte Carlo algorithms may err with bounded probability.
- TF15:** False - Reservoir sampling is designed for unknown stream length.
- TF16:** True - The increment probability decreases with k .
- TF17:** True - Standard insertion-only Bloom filters have no false negatives.
- TF18:** False - Bloom filters can produce false positives.
- TF19:** True - Collisions can only increase counters, biasing estimates upward.
- TF20:** True - Taking the minimum across rows reduces collision impact.

Mock Exam 2

A) Multiple Choice (20 questions)

MCQ1 [Medium] A recursion tree (as used in your recurrence discussion) is mainly a tool to:

- A) Prove NP-completeness
- B) Visualise repeated subcalls and estimate total work across recursion levels
- C) Remove false positives in Bloom filters
- D) Store stream elements explicitly

MCQ2 [Medium] Memoization is most naturally associated with which pattern?

- A) "Compute once, reuse many times" for overlapping subcalls
- B) "Sort once, query forever"
- C) "Hash once, no collisions"
- D) "Choose locally best, always optimal"

MCQ3 [Hard] In DP examples (e.g., Coin Row / Knapsack), the phrase optimal substructure means:

- A) The optimal solution can be written as a combination of optimal solutions to subproblems
- B) Subproblems never repeat
- C) Randomness is required to avoid worst cases
- D) The solution space is always a tree

MCQ4 [Medium] For 0-1 Knapsack, a key reason DP can be expensive on large instances is:

- A) DP is always exponential by definition
- B) DP may depend on capacity/weights in a way that makes runtime scale with numeric values (pseudo-polynomial behaviour)
- C) DP requires universal hashing
- D) DP can only be implemented with backtracking

MCQ5 [Medium] Activity Selection is typically solved by:

- A) Picking activities with the smallest duration first
- B) Picking activities with the earliest finishing time first
- C) Picking activities with the largest start time first
- D) Picking activities at random and repeating

MCQ6 [Hard] A classic pitfall highlighted in greedy lectures is that:

- A) Greedy always fails, so we never use it
- B) Greedy can be correct for some problems (e.g., activity selection) but fail for others (e.g., 0-1 knapsack)
- C) Greedy can be made correct by adding a Bloom filter
- D) Greedy is identical to DP with a smaller table

MCQ7 [Medium] In randomized quicksort, randomisation is used mainly to:

- A) Make the algorithm always $O(n)$
- B) Reduce the chance of consistently bad pivot choices, improving expected performance
- C) Remove the need for comparisons
- D) Guarantee stability

MCQ8 [Hard] A key difference between Monte Carlo and Las Vegas methods is:

- A) Only Las Vegas uses random bits
- B) Only Monte Carlo can be used in streaming
- C) Monte Carlo may trade correctness probability for speed; Las Vegas trades runtime randomness for correctness
- D) They are the same; different names only

MCQ9 [Medium] The birthday paradox intuition mainly explains why:

- A) Exact counting is impossible
- B) Collisions appear sooner than linear scale; the threshold is around square-root of the space size
- C) MSTs require cycle checks
- D) Bloom filters have false negatives

MCQ10 [Medium] The coupon collector intuition mainly supports the claim that:

- A) Collecting all types takes more than linear time; growth is roughly $n \log n$ order
- B) Greedy is optimal for 0-1 knapsack
- C) Reservoir sampling is biased
- D) Count-Min always underestimates

MCQ11 [Hard] In the probabilistic counter with arbitrary base a , if you increase a :

- A) The increment probability $1/a^k$ increases for fixed k
- B) The increment probability $1/a^k$ decreases for fixed k
- C) The algorithm becomes deterministic
- D) Bloom filter false positives become zero

MCQ12 [Medium] Bloom filter membership query procedure:

- A) Check one cell only; if it's 1 return "in set"
- B) Check all k hashed positions; if any is 0 - definitely not in set; else - maybe in set
- C) Count frequency of the element and compare to threshold
- D) Take minimum across rows

MCQ13 [Hard] Bloom filter optimal k is approximately:

- A) $k \approx (m/n) \ln 2$
- B) $k \approx (n/m) \ln 2$

- C) $k \approx \ln(mn)$
- D) $k \approx m - n$

MCQ14 [Medium] A key feature of Cuckoo filters is:

- A) They cannot support deletions
- B) They store fingerprints and can support deletions (unlike standard Bloom)
- C) They require storing the full elements
- D) They eliminate all false positives

MCQ15 [Medium] Count-Min Sketch update conceptually does:

- A) Decrement all counters when full
- B) Increment one counter total
- C) Increment one counter per row (one per hash function)
- D) Replace a stored fingerprint at random

MCQ16 [Hard] Count-Min Sketch estimates are typically:

- A) Underestimates due to min operator
- B) Overestimates due to collisions; min reduces but doesn't remove overcounting
- C) Exact, because multiple hash functions remove collisions
- D) Random and unbiased around the truth

MCQ17 [Hard] In your Count-Min analysis, Markov's inequality is used to argue:

- A) The median is always exact
- B) The chance of the collision error being much larger than its expectation is bounded
- C) Bloom filters have no false positives
- D) MST edges must be sorted

MCQ18 [Medium] Misra-Gries maintains:

- A) A full frequency table for all items
- B) A small associative array of at most $k-1$ counters (plus a decrement step when full)
- C) A matrix of counters indexed by hash functions
- D) A set of hash fingerprints for membership only

MCQ19 [Medium] For distinct counting methods (FM/HyperLogLog ideas), the trailing zeros idea is used because:

- A) Trailing zeros in hashed outputs are correlated with the number of distinct elements seen
- B) It gives exact distinct counts with no variance
- C) It works only if the stream is sorted
- D) It removes collisions in MST construction

MCQ20 [Hard] HyperLogLog is best thought of as:

- A) A deterministic exact distinct counter
- B) A practical variant that uses registers/buckets to reduce variance in distinct counting
- C) A greedy MST algorithm
- D) A DP algorithm for coin selection

B) True/False (20 statements)

TF1 [Medium] A recursion tree is useful to estimate total work by summing costs across recursion levels.

TF2 [Medium] Memoization is mainly useful when subproblems repeat.

TF3 [Hard] Optimal substructure means subproblems never overlap.

TF4 [Medium] Activity selection can be solved greedily by choosing the activity with the earliest finishing time compatible with the schedule.

TF5 [Medium] Greedy is guaranteed to solve 0-1 knapsack optimally if you sort by value/weight ratio.

TF6 [Medium] Randomised quicksort uses randomisation to improve expected performance, not to change the worst-case bound.

TF7 [Medium] Monte Carlo algorithms may be incorrect with some probability.

TF8 [Medium] Las Vegas algorithms may have random runtime but are correct when they return an answer.

TF9 [Medium] The birthday paradox suggests collisions happen around $O(\sqrt{N})$ samples (order-of-magnitude).

TF10 [Medium] Coupon collector suggests collecting all coupon types takes roughly $O(n \log n)$ (order-of-magnitude).

TF11 [Hard] In approximate counting with base a , increasing a increases the chance of increment for fixed k .

TF12 [Medium] Bloom filters answer "definitely not in set" if any hashed bit is 0.

TF13 [Hard] Bloom filters can be tuned by choosing m and k to reduce false positives.

TF14 [Medium] Cuckoo filters are based on storing fingerprints rather than setting bits only.

TF15 [Hard] Count-Min Sketch returns the maximum across rows to reduce collisions.

TF16 [Medium] Count-Min Sketch uses multiple hash functions and multiple counter arrays (one per hash function).

TF17 [Hard] Markov's inequality can be used to bound how often the collision error greatly exceeds its expected value.

TF18 [Medium] Misra-Gries uses a decrement-all step when its counter set is full and a new unseen item arrives.

TF19 [Medium] Trailing zeros are computed on hashed values, not on the raw stream tokens.

TF20 [Hard] HyperLogLog reduces variance by splitting the stream into buckets/registers and combining their information.

C) Answer Key

MCQ: 1B, 2A, 3A, 4B, 5B, 6B, 7B, 8C, 9B, 10A, 11B, 12B, 13A, 14B, 15C, 16B, 17B, 18B, 19A, 20B

T/F: 1T, 2T, 3F, 4T, 5F, 6T, 7T, 8T, 9T, 10T, 11F, 12T, 13T, 14T, 15F, 16T, 17T, 18T, 19T, 20T

D) Brief Explanations

MCQ1: B - Recursion trees help sum work across levels; they are not about NP-completeness or AMQs.

MCQ2: A - Memoization follows 'compute once, reuse' when subcalls overlap.

MCQ3: A - Optimal substructure means optimal solutions are composed of optimal subproblem solutions.

MCQ4: B - 0-1 knapsack DP can be pseudo-polynomial, scaling with numeric capacity/weights.

MCQ5: B - Earliest finishing time is the classic optimal greedy rule.

MCQ6: B - Greedy is correct for some problems and fails for others; proof matters.

MCQ7: B - Random pivots avoid adversarial sequences, improving expected performance.

MCQ8: C - Monte Carlo risks error with bounded runtime; Las Vegas is correct but runtime random.

MCQ9: B - Collisions arise around sqrt-space due to birthday paradox.

MCQ10: A - Coupon collection takes superlinear time, about $n \log n$ order.

MCQ11: B - Increasing a decreases $1/a^k$, so increments become rarer.

MCQ12: B - Bloom membership: any 0 means absent; all 1 means 'maybe'.

MCQ13: A - $k_{\text{opt}} \approx (m/n) \ln 2$.

MCQ14: B - Cuckoo stores fingerprints and supports deletions.

MCQ15: C - Count-Min increments one counter in each row via different hashes.

MCQ16: B - Collisions inflate counts; min reduces but cannot remove inflation.

MCQ17: B - Markov bounds probability that error greatly exceeds expectation.

MCQ18: B - Misra-Gries maintains up to $k-1$ counters and decrements on overflow.

MCQ19: A - Trailing zeros in hashed outputs correlate with distinct count.

MCQ20: B - HyperLogLog uses registers/buckets to reduce variance.

TF1: True - Summing level costs is the recursion-tree technique.

TF2: True - Memoization helps when subproblems repeat.

TF3: False - That's overlapping subproblems, not optimal substructure.

TF4: True - Earliest finish time greedy is correct for activity selection.

TF5: False - Ratio-greedy fails for 0-1 knapsack.

TF6: True - Randomisation improves expected behaviour; worst case remains $O(n^2)$.

TF7: True - Monte Carlo can be wrong with some probability.

TF8: True - Las Vegas is correct when it returns.

TF9: True - Collision threshold is around sqrt scale.

TF10: True - Expected time to collect all types is about $n \log n$ order.

TF11: False - Increasing a lowers increment probability.

TF12: True - Any 0 bit means definitely absent.

TF13: True - m and k can be tuned to reduce false positives.

TF14: True - Fingerprints are the key idea.

TF15: False - Count-Min takes the minimum, not the maximum.

TF16: True - Multiple hash functions and rows are used.

TF17: True - Markov helps bound large deviations from expectation.

TF18: True - Decrement-all step is how it makes room.

TF19: True - Computed on hashed representations.

TF20: True - Buckets/registers reduce variance.

Mock Exam 3

A) Multiple Choice (20 questions)

MCQ1 [Medium] The Misra-Gries algorithm maintains at most:

- A) 1 counter
- B) $k-1$ counters
- C) k counters per stream element
- D) A full counter per distinct element

MCQ2 [Hard] In Misra-Gries, when the associative array is full and a new unseen token arrives, the algorithm:

- A) Ignores the token and continues
- B) Decrements all counters and removes any that drop to 0
- C) Rehashes all elements into a bigger table
- D) Returns "maybe present"

MCQ3 [Medium] The key reason Misra-Gries is useful in streams is that it:

- A) Gives exact frequencies for all tokens with sublinear memory
- B) Tracks a small set of candidate frequent items with limited memory
- C) Eliminates collisions by sorting the stream
- D) Produces exact distinct counts

MCQ4 [Hard] Count-Min Sketch differs from a counting Bloom filter mainly because:

- A) Count-Min is designed for approximate per-item frequency queries; counting Bloom is designed to represent multiplicities/membership-style counts
- B) Count-Min guarantees no collisions; counting Bloom does not
- C) Counting Bloom is sublinear; Count-Min always stores all items
- D) They are identical data structures

MCQ5 [Medium] In Count-Min Sketch, the usual query estimate is the:

- A) Max across rows
- B) Min across rows
- C) Median across rows
- D) Sum across rows

MCQ6 [Hard] Count-Min Sketch estimates typically:

- A) Underestimate true frequencies
- B) Overestimate true frequencies
- C) Are exactly correct if $d \geq 2$
- D) Are unbiased around true frequencies

MCQ7 [Hard] In Count-Min analysis, if expected collision noise per row is about N/w , increasing w generally:

- A) Increases expected collision noise
- B) Decreases expected collision noise
- C) Doesn't change anything
- D) Makes the sketch deterministic

MCQ8 [Hard] The role of taking multiple rows (multiple hash functions) in Count-Min is to:

- A) Make the estimate exactly correct
- B) Allow taking a minimum so at least one row likely has low collision noise
- C) Remove the need for hashing
- D) Replace DP in knapsack

MCQ9 [Medium] A standard Bloom filter false positive happens when:

- A) At least one hashed position is 0
- B) All hashed positions are 1 due to other inserted elements
- C) The element was inserted twice
- D) You forgot to choose k

MCQ10 [Hard] The fraction of cells set to 1 idea in Bloom filters is used because:

- A) It directly determines false-negative probability
- B) False-positive probability is closely related to how many bits are 1 and to k
- C) It removes the need for hash functions
- D) It guarantees exact membership

MCQ11 [Hard] Cuckoo filters are typically described as:

- A) Bit arrays with no deletions
- B) Fingerprint-based structures with better practicality for deletions and sometimes space compared to Bloom
- C) Exact frequency estimators
- D) MST solvers

MCQ12 [Medium] In approximate counting with decreasing probability, the probability of increment generally:

- A) Increases as the counter increases
- B) Decreases as the counter increases
- C) Stays constant
- D) Becomes 0 immediately after the first increment

MCQ13 [Hard] For the arbitrary-base method, the estimate $(a^k - a + 1)/(a - 1)$ is designed to:

- A) Exactly equal the true count after every run

- B) Map a small stored counter value to an approximate large event count
- C) Compute a shortest path
- D) Compute an MST

MCQ14 [Medium] The trailing zeros concept in distinct counting is computed from:

- A) Raw integers in the stream without hashing
- B) Hashed values of stream elements expressed in binary
- C) Edge weights of a graph
- D) DP tables

MCQ15 [Hard] The intuition behind trailing zeros in FM/HyperLogLog methods is that:

- A) Long runs of trailing zeros are more likely when you've seen more distinct elements
- B) Trailing zeros are equally likely regardless of number of distincts
- C) It produces exact results
- D) It only works for sorted streams

MCQ16 [Medium] Approximate solutions / heuristics (e.g., for TSP) are justified because:

- A) Real-world data is always exact, so approximation is unnecessary
- B) Exact solutions to hard optimisation problems may be too slow; heuristics can be good enough
- C) DP is always faster than greedy
- D) NP-complete problems are impossible to approximate

MCQ17 [Hard] A statement consistent with NP-completeness discussion is:

- A) Backtracking always solves NP-complete problems in polynomial time
- B) NP-completeness is shown by polynomial-time reductions from known NP-complete problems (e.g., SAT)
- C) Bloom filters prove NP-completeness
- D) Count-Min Sketch can solve SAT approximately

MCQ18 [Medium] In greedy MST algorithms, the cycle-avoidance check is essential because:

- A) Cycles lower the total weight automatically
- B) A spanning tree by definition has no cycles; adding a cycle breaks the structure
- C) Cycles are required for optimality
- D) Cycles are removed by memoization

MCQ19 [Medium] Reservoir sampling is best used to:

- A) Select a uniform sample from a stream without knowing stream length in advance
- B) Compute a DP solution faster
- C) Store all stream elements explicitly
- D) Eliminate false positives in Bloom filters

MCQ20 [Hard] Randomness in algorithms is used to:

- A) Replace correctness proofs
- B) Guide choices so behaviour is less dependent on worst-case/adversarial inputs
- C) Guarantee the same runtime on every input
- D) Avoid hash functions entirely

B) True/False (20 statements)

TF1 [Medium] Misra-Gries keeps at most $k-1$ keys in its associative array.

TF2 [Hard] In Misra-Gries, when full and a new unseen item arrives, the algorithm decrements all counters and deletes zeros.

TF3 [Hard] Misra-Gries stores the entire stream to be accurate.

TF4 [Medium] Count-Min Sketch uses multiple hash functions and multiple rows of counters.

TF5 [Hard] Count-Min Sketch can return an underestimate because it takes a minimum.

TF6 [Medium] Count-Min Sketch estimates can be inflated by collisions.

TF7 [Hard] Increasing w in Count-Min tends to reduce expected collision noise per row.

TF8 [Hard] Markov inequality can bound the probability that collision noise is far above its expectation.

TF9 [Medium] Bloom filters can have false positives.

TF10 [Medium] Standard Bloom filters have false negatives in insertion-only mode.

TF11 [Medium] Bloom filters use k hash functions and check all k positions for membership queries.

TF12 [Hard] The optimal k for a Bloom filter is approximately proportional to m/n .

TF13 [Medium] Cuckoo filters are fingerprint-based.

TF14 [Hard] Approximate counting with base a increments with probability $1/a^k$ when the counter is k .

TF15 [Medium] The estimator $(a^k - a + 1)/(a - 1)$ maps a small counter value to a larger approximate count.

TF16 [Medium] Distinct counting methods compute properties (like trailing zeros) of hashed values.

TF17 [Hard] HyperLogLog reduces variance by using multiple registers/buckets.

TF18 [Medium] Heuristics for TSP are motivated by the impracticality of exact solutions on large instances.

TF19 [Medium] NP-completeness arguments often use SAT as a starting point for reductions.

TF20 [Hard] Randomised quicksort's worst-case time bound improves from $O(n^2)$ to $O(n \log n)$ due to randomisation.

C) Answer Key

MCQ: 1B, 2B, 3B, 4A, 5B, 6B, 7B, 8B, 9B, 10B, 11B, 12B, 13B, 14B, 15A, 16B, 17B, 18B, 19A, 20B

T/F: 1T, 2T, 3F, 4T, 5F, 6T, 7T, 8T, 9T, 10F, 11T, 12T, 13T, 14T, 15T, 16T, 17T, 18T, 19T, 20F

D) Brief Explanations

- MCQ1:** B - Misra-Gries keeps at most $k-1$ counters.
- MCQ2:** B - When full and a new item arrives, it decrements all counters and removes zeros.
- MCQ3:** B - It tracks a small set of candidate frequent items using limited memory.
- MCQ4:** A - Count-Min targets approximate frequency queries; counting Bloom is membership/multiset style.
- MCQ5:** B - Count-Min returns the minimum across rows.
- MCQ6:** B - Collisions can only increase counts, so estimates overcount.
- MCQ7:** B - Larger width w reduces expected collision noise per row.
- MCQ8:** B - Multiple rows allow the minimum to likely come from a low-collision row.
- MCQ9:** B - False positives occur when all k bits are set due to other items.
- MCQ10:** B - False positives depend on fraction of 1 bits and k .
- MCQ11:** B - Cuckoo uses fingerprints and supports deletions; it can be space-competitive.
- MCQ12:** B - Increment probability decreases as the counter grows.
- MCQ13:** B - Estimator decodes small k into a large approximate count.
- MCQ14:** B - Trailing zeros are computed on hashed binary values.
- MCQ15:** A - With more distinct items, rarer hash patterns (more trailing zeros) become more likely.
- MCQ16:** B - Exact solutions may be too slow; heuristics give practical approximations.
- MCQ17:** B - NP-completeness is shown via polynomial-time reductions (often from SAT).
- MCQ18:** B - Spanning trees must be acyclic; cycle checks enforce that.
- MCQ19:** A - Reservoir sampling yields a uniform sample without knowing stream length.
- MCQ20:** B - Randomness reduces dependence on adversarial inputs; it does not replace proofs.
- TF1:** True - Misra-Gries keeps at most $k-1$ keys.
- TF2:** True - Decrement-all and delete zeros is the overflow rule.
- TF3:** False - It does not store the stream.
- TF4:** True - Multiple rows/counters are core to Count-Min.
- TF5:** False - The estimate is an overestimate; min selects the least inflated one.
- TF6:** True - Collisions inflate counts.
- TF7:** True - Increasing w spreads collisions, reducing expected noise.
- TF8:** True - Markov bounds large deviations from expectation.
- TF9:** True - Bloom filters can return false positives.
- TF10:** False - Standard insertion-only Bloom has no false negatives.
- TF11:** True - It checks all k hashed positions.
- TF12:** True - $k_{\text{opt}} \approx (m/n) \ln 2$ so proportional to m/n .
- TF13:** True - Fingerprints are central to cuckoo filters.
- TF14:** True - Increment probability is $1/a^k$.
- TF15:** True - The estimator decodes to an approximate large count.
- TF16:** True - These methods operate on hashed representations.
- TF17:** True - HyperLogLog reduces variance via many registers/buckets.

TF18: True - Heuristics are motivated by practicality on large instances.

TF19: True - SAT is a canonical base problem for reductions.

TF20: False - Randomisation improves expected time, not worst-case bound.